Aula 7 - Atributos e Métodos da Classe: Marcador *Static*

Programação Orientada a Objetos
Prof. Matheus Avila Moreira de Paula
matheusavila@cefetmg.br

Engenharia de Computação
Centro Federal de Educação Tecnológica Leopoldina

Static

```java
public class Carro {
    private double preco;
    private String modelo;
    private String marca;
    private int numPortas;

    public double getPreco() {
        return preco;}

    public void setPreco(double preco) {
        this.preco = preco;}

    public String getModelo() {
        return modelo;}

    public void setModelo(String modelo) {
        this.modelo = modelo;}

    public String getMarca() {
        return marca;}

    public void setMarca(String marca) {
        this.marca = marca;}

    public int getNumPortas() {
        return numPortas;}

    public void setNumPortas(int numPortas) {
        this.numPortas = numPortas;}

}
```

Static

- Suponhamos agora que a concessionária quer controlar a quantidade de carros cadastrados no sistema. Qual a forma mais fácil de fazer isso?
  - Podemos utilizar as estruturas HashSet, ArrayList ou Map
  - Podemos fazer um contador no main()

Static

- Seria conveniente que essa informação fosse única, centralizada e compartilhada por todos os objetos dessa classe
  - Quando fosse alterado através de um objeto, os outros enxergam o mesmo valor
  - Para fazer isso em Java, declaramos o atributo como estático (static).
- Atributos e Métodos com o modificador static pertencem a **classe** e não a um objeto específico.

Static

- Atributos e métodos estáticos são **compartilhados** por todas as instâncias da classe
- Em um atributo estático, somente um valor será armazenado, e caso o valor seja alterado, a modificação será refletida em todas as demais instâncias
- O modificador static pode ser combinado com modificadores como public, protected e private

Static

- Atributos e Métodos sem o modificador static pertencem ao objeto instanciado
- Atributos e Métodos com o modificador static pertencem a classe

Static

- Podemos alterar um atributo static em qualquer função da classe
- No caso de contar o número de carros essa contagem é feita no construtor

Static

```java
public class ExercStatic {

    public static void main(String[] args) {

        Carro c1 = new Carro();
        Carro c2 = new Carro();
        Carro c3 = new Carro();

        System.out.println(Carro.numCarros);


    }
}
```

```java
public class Carro {
    private double preco;
    private String modelo;
    private String marca;
    private int numPortas;

    public static int numCarros;
    public Carro(){
        numCarros++;}

    public double getPreco() {
        return preco;}

    public void setPreco(double preco) {
        this.preco = preco;}

    public String getModelo() {
        return modelo;}

    public void setModelo(String modelo) {
        this.modelo = modelo;}

    public String getMarca() {
        return marca;}

    public void setMarca(String marca) {
        this.marca = marca;}

    public int getNumPortas() {
        return numPortas;}

    public void setNumPortas(int numPortas) {
        this.numPortas = numPortas;}

}
```

Static

- Infelizmente, deixar o atributo numCarros public permite que ele seja alterado livremente a qualquer momento no código.

```java
public class ExercStatic {

    public static void main(String[] args) {

        Carro c1 = new Carro();
        Carro c2 = new Carro();
        Carro c3 = new Carro();

        System.out.println(Carro.numCarros);

        Carro.numCarros = 15;

        System.out.println(Carro.numCarros);

    }
}
```

Static

- Para corrigir essa falha precisamos alterar o atributo static numCarros para private.
- Agora, para acessar esse atributo precisamos definir um método static.

```
public class ExercStatic {

    public static void main(String[] args) {

        Carro c1 = new Carro();
        Carro c2 = new Carro();
        Carro c3 = new Carro();


        System.out.println(Carro.getNumCarros());

    }
}
```

Static

- Se removermos o termo static do método getNumCarros() a leitura da quantidade de carros poderia ser feita?
  - Sim! Mas isso não está de acordo com o propósito de um atributo static, pois agora, é necessário ter um objeto instanciado para acessar uma variável que pertence a classe

Static

```java
public class ExercStatic {

    public static void main(String[] args) {

        Carro c1 = new Carro();
        Carro c2 = new Carro();
        Carro c3 = new Carro();


        System.out.println(c1.getNumCarros());

    }
}
```

```java
public class Carro {
    private double preco;
    private String modelo;
    private String marca;
    private int numPortas;

    private static int numCarros;

    public int getNumCarros(){
        return numCarros;
    }
```

Static

- Além disso, podemos fazer um método para a venda de um carro. Dessa forma, o contador de carros deve ser decrementado sempre que um carro for vendido

```java
public class ExercStatic {

    public static void main(String[] args) {

        Carro c1 = new Carro();
        Carro c2 = new Carro();
        Carro c3 = new Carro();


        System.out.println(Carro.getNumCarros());

        c1.vendaCarro();
        System.out.println(Carro.getNumCarros());


    }
}
```

```java
public class Carro {
    private double preco;
    private String modelo;
    private String marca;
    private int numPortas;

    private static int numCarros;

    public static int getNumCarros(){
        return numCarros;
    }

    public void vendaCarro(){
        numCarros--;
    }

    public Carro(){
        numCarros++;}
```

Exercício

- Crie uma classe atendente de banco. Esta classe deve ter como atributos:
  - Contador do número de atendentes, identificador incremental inteiro gerado para cada atendente instanciado, Nome, disponibilidade do atendente (ocupado ou livre), número de pessoas na fila ou sendo atendidas para cada atendente.
  - Defina uma função static no *main* para colocar uma pessoa na fila de um atendente baseado em seu identificador.
  - Defina um método não static para remover uma pessoa da fila de determinado caixa.
  - Defina um método para liberar um atendente. Para ser liberado, um atendente não pode estar ocupado, ou seja, não ter ninguém na fila.